THE UNIVERSITY OF MELBOURNE

COMP30023 – Computer Systems

Interprocess Communication

Maria Rodriguez Read

Semester 1, 2025

© University of Melbourne

Interprocess Communication (IPC)

- Processes frequently need to communicate with other processes. They need to share resources/data
- Issues:
  - How? (mechanisms for IPC: shared memory, pipes, files, sockets, signals…)
  - Cooperating processes may interfere with each other and require an orderly execution
- **Our focus:** understanding and avoiding **race conditions** in cooperating processes

© University of Melbourne

Process vs. Thread

- Process/thread used interchangeably in this lecture
- Why?
  - The same problems and the same solutions apply to both

Race Conditions

- Occur when multiple processes/threads access a shared resource (e.g., a shared variable)
- The results depend on the timing of the code's execution – the order in which instructions are executed
- Results are indeterminate. The outcome is not known and is likely to be different across runs

© University of Melbourne

Example of Race Condition

Pseudo code for a function popping from a stack

```
Stack aStack; //global variable
...
void  myStackFunction()
{
    if (isEmpty(aStack))
    {
        return;
    }

    int element = pop(aStack);

    //do something with element
}
```

shared resource

executed as a thread

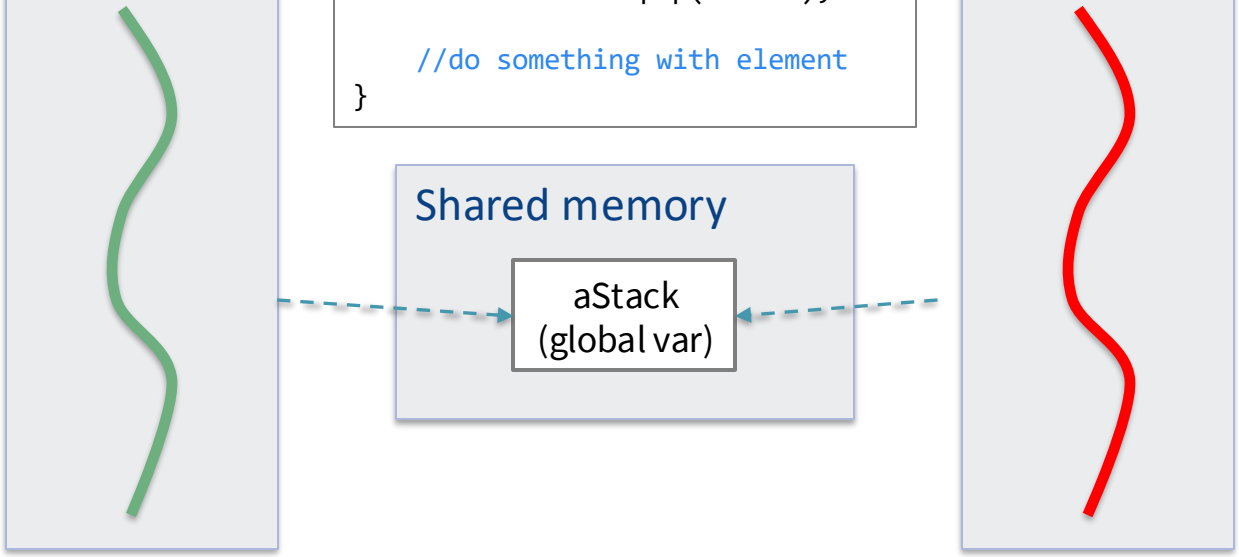
this line of code should never run if the stack is empty

Example of Race Condition

```
Stack aStack; //global variable
...
void  myStackFunction()
{
    if (isEmpty(aStack))
    {
        return;
    }

    int element = pop(aStack);

    //do something with element
}
```



Example of Race Condition

```
Stack aStack; //global variable
...
void  myStackFunction()
{
    if (isEmpty(aStack))
    {
        return;
    }

    int element = pop(aStack);

    //do something with element
}
```

CPU allocation

Execution Transcript

(assume only 1 element in the stack)

1. Thread A tests `isEmpty(aStack)`⇒ false
2. Thread A pops ⇒ stack is now empty
3. Thread B tests `isEmpty(aStack)`⇒ true
4. Thread B just returns – nothing to do

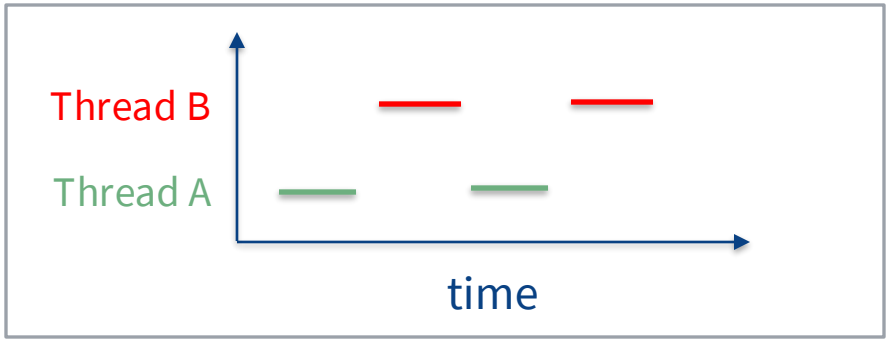
Example of Race Condition

```
Stack aStack; //global variable
...
void  myStackFunction()
{
    if (isEmpty(aStack))
    {
        return;
    }

    int element = pop(aStack);

    //do something with element
}
```

CPU allocation



Execution Transcript

(assume only 1 element in the stack)

1. Thread A tests `isEmpty(aStack)`⇒ false
2. Thread B tests `isEmpty(aStack)`⇒ false
3. Thread A pops ⇒ aStack is now empty
4. Thread B pops – program logic was designed for this to never happen!

Example of Race Condition

- Demonstration of race condition in multithreaded program

Avoiding Race Conditions

- **Critical Region:** Part of the program where shared resources are accessed
- **Mutual Exclusion:** Guarantee that only 1 process/thread executes within the critical region at any given point in time

© University of Melbourne

Example of Critical Region

Pseudocode for a function popping from a stack

```
Stack aStack; //global variable
...
void  myStackFunction()
{
    if (isEmpty(aStack))
    {
        return;
    }

    int element = pop(aStack);

    //do something with element
}
```

critical region

Example of Mutual Exclusion

If the critical region for our stack example had the property of mutual exclusion, then

Execution Transcript 1

1. Thread A tests `isEmpty(aStack)`⇒ false
2. Thread A pops ⇒ stack is now empty
3. Thread B tests `isEmpty(aStack)`⇒ true
4. Thread B just returns – nothing to do

Execution Transcript 2

1. Thread A tests `isEmpty(aStack)`⇒ false
2. Thread B tests `isEmpty(aStack)`⇒ false
3. Thread A pops ⇒ stack is now empty
4. Thread B pops – program logic was designed for this to never happen!

© University of Melbourne

Example of Mutual Exclusion

If the critical region for our stack example had the property of mutual exclusion, then execution transcript 2 would never occur

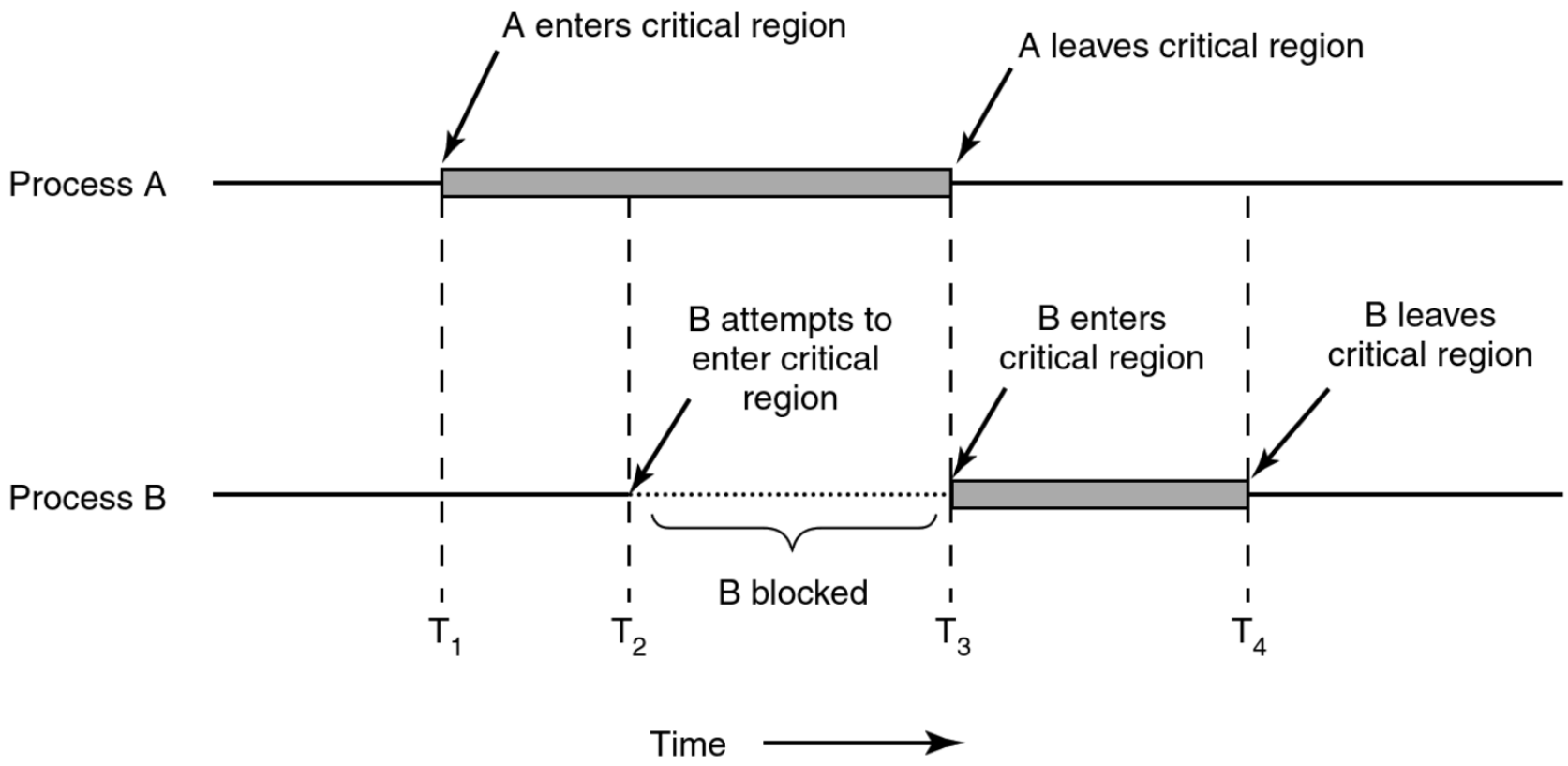
Execution Transcript 1

1. Thread A tests `isEmpty(aStack)`⇒ false
2. Thread A pops ⇒ stack is now empty
3. Thread B tests `isEmpty(aStack)`⇒ true
4. Thread B just returns – nothing to do

Execution Transcript 2

1. Thread A tests `isEmpty(aStack)`⇒ false
2. Thread B tests `isEmpty(aStack)`⇒ false
3. Thread A pops ⇒ stack is now empty
4. Thread B pops – program logic was designed for this to never happen!

Mutual Exclusion Visualized



TB 2-22

Conditions for Mutual Exclusion

Conditions to achieve **a good solution**:

1. No two processes may be simultaneously inside their critical regions (*mutual exclusion*)
2. No assumptions may be made about speeds or the number of CPUs
3. No process running outside its critical region may block other processes
4. No process should have to wait forever to enter its critical region (*starvation*)

© University of Melbourne

Achieving Mutual Exclusion

- General idea: use locks to control access to the critical region
- **Acquire lock** to enter the critical region
- **Release lock** when exiting the critical region
- Only the process/thread with the lock can be in the critical region

© University of Melbourne

Lock Variables

A possible implementation using locks would be:

```
int lock; //global variable
...
void  threadFunction()
{
    while (lock != 0){
    }
    lock = 1;
    criticalRegion();
    lock = 0;
    //non-critical region
}
```

busy waiting – spin until lock becomes 0 (is released)

acquire lock

release lock

Lock Variables

```
int lock; //global variable
...
void  threadFunction()
{
    while (lock != 0){
    }
    lock = 1;
    criticalRegion();
    lock = 0;
    //non-critical region
}
```

Context switch right after exiting from loop, just before executing this instruction

A *context switch* is the mechanism by which the operating system stops the execution of one thread or process and starts the execution of another thread or process

Lock Variables

If the execution of a thread is stopped right after exiting the loop and just before setting the lock to 1:

```
int lock; //global variable
...
void  threadFunction()
{
    while (lock != 0){
    }
    lock = 1;
    criticalRegion();
    lock = 0;
    //non-critical region
}
```

Execution Transcript

(assume 2 threads executing threadFunction())

1. Thread A tests lock != 0 ⇒ false, exits loop (lock is 0)
2. Thread B tests lock != 0 ⇒ false, exits loop (lock is 0)
3. Thread A executes lock = 1 (lock is 1), enters the critical region
4. Thread B executes lock = 1 (lock is 1), enters the critical region

We now have 2 threads executing the critical region!

Mutual Exclusion

- Busy Waiting
  - Strict alternation
  - TSL with busy waiting
- Blocking
  - Mutex

© University of Melbourne

Busy Waiting

General approach

When a process wants to enter the critical region

- It checks if entry is allowed
- If not, the process executes a **loop**, continuously checking if it is allowed to enter the critical region

Cons:

- Waste of CPU
- Priority Inversion Problem

© University of Melbourne

Priority Inversion Problem

- Consider a computer running 2 processes:
  - P_{high} has high priority
  - P_{low} has low priority
- Scheduling rules: P_{high} always gets to run as long as it is not *blocked*
- At a particular moment:
  - P_{low} is in the critical region
  - P_{high} becomes ready to run (e.g., I/O operation completed)
  - P_{high} starts running and begins busy waiting (can't enter critical region because P_{low} is in it)
  - P_{low} never gets to exit the critical region because it never gets scheduled to use the CPU while P_{high} is running
  - P_{high} loops forever

© University of Melbourne

Strict Alternation

Busy waiting approach in which 2 processes/threads take turns running the critical region

Thread A gets to enter the critical region when turn is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when turn is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

Strict Alternation

Thread A gets to enter the critical region when *turn* is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when *turn* is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

turn = 0

Strict Alternation

Thread A gets to enter the critical region when *turn* is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when *turn* is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

turn = 0

Strict Alternation

Thread A gets to enter the critical region when *turn* is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when *turn* is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

turn = 0

Strict Alternation

Thread A gets to enter the critical region when *turn* is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when *turn* is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

```
turn = 1
```

Strict Alternation

Thread A gets to enter the critical region when *turn* is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when *turn* is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

```
turn = 1
```

Strict Alternation

Thread A gets to enter the critical region when *turn* is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when *turn* is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

```
turn = 1
```

Strict Alternation

Thread A gets to enter the critical region when *turn* is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when *turn* is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

```
turn = 1
```

Strict Alternation

Thread A gets to enter the critical region when *turn* is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when *turn* is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

turn = 0

Strict Alternation

Thread A gets to enter the critical region when *turn* is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when *turn* is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

```
turn = 0
```

Strict Alternation

Thread A gets to enter the critical region when *turn* is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when *turn* is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

turn = 0

Strict Alternation

Thread A gets to enter the critical region when *turn* is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when *turn* is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

```
turn = 1
```

Strict Alternation

Thread A gets to enter the critical region when *turn* is 0:

```
int turn; //global variable
...
void  threadA()
{
  while(TRUE) {
    while (turn != 0){}
    criticalRegionA();
    turn = 1;
    nonCriticalRegionA();
  }
}
```

Thread B gets to enter the critical region when *turn* is 1:

```
int turn; //global variable
...
void  threadB()
{
  while(TRUE) {
    while (turn != 1){}
    criticalRegionB();
    turn = 0;
    nonCriticalRegionB();
  }
}
```

```
turn = 1
```

Thread B is blocking Thread A from entering the critical region, but Thread B is outside of the critical region

Test and Set Lock (TSL)

- Hardware support for locking
- **Test and Set** Lock (TSL) instruction

```
TSL REGISTER, LOCK
```

1. Copy contents of LOCK into REGISTER
2. Store non-zero value in LOCK

Atomic Operations

- Atomic operations are indivisible, all or nothing

Mutual Exclusion with TSL and Busy Waiting

To enter the critical region:

1. TSL REGISTER, LOCK
2. If REGISTER is 0, enter critical region
3. If REGISTER is not 0, lock is set so go back to step 1 (busy wait)

When leaving the critical region:

1. Set lock to 0

© University of Melbourne

Mutual Exclusion with TSL and Busy Waiting

```
enter_region:
        TSL REGISTER,LOCK          | copy lock to register and set lock to 1
        CMP REGISTER,#0            | was lock zero?
        JNE enter_region           | if it was not zero, lock was set, so loop
        RET                        | return to caller; critical region entered

leave_region:
        MOVE LOCK,#0               | store a 0 in lock
        RET                        | return to caller
```

TB 2-25

Blocking

General approach

When a thread wants to enter its critical region

- It checks if entry is allowed
- If not, let another thread use the CPU while waiting for the lock
- How? E.g., block, yield

Cons:

- Approach-specific (e.g., *starvation*, overhead of system calls, overhead of *context switches*)

Yield Mutex

```
mutex_lock:
        TSL REGISTER,MUTEX      | copy mutex to register and set mutex to 1
        CMP REGISTER,#0         | was mutex zero?
        JZE ok                  | if it was zero, mutex was unlocked, so return
        CALL thread_yield       | mutex is busy; schedule another thread
        JMP mutex_lock          | try again
ok:     RET                     | return to caller; critical region entered

mutex_unlock:
        MOVE MUTEX,#0           | store a 0 in mutex
        RET                     | return to caller
```

TB 2-29

Deadlocks

Process A

```
Lock(M1)
DoWork(…)
Lock(M2)
DoWork(…)
Release_lock(M1)
Release_lock(M2)
```

Process B

```
Lock(M2)
DoWork(…)
Lock(M1)
DoWork(…)
Release_lock(M2)
Release_lock(M1)
```

A set of processes is *deadlocked* if each process in the set is waiting for an event that only another process in the set can cause

May exist in many shared resource settings; not just memory

Acknowledgements

- The slides were prepared by Maria Rodriguez Read based on material developed previously by Olya Ohrimenko.

© University of Melbourne